

ESPRIT AI Receptionist

Rapport de suivi de projet – Assistant vocal intelligent

Stage – ESPRIT

Document vivant, mis à jour au fil du projet
Dernière mise à jour : 20 juillet 2026

Contents

1	Introduction et contexte	3
1.1	Objectif du projet	3
1.2	Contrainte fondamentale : 100% gratuit et local	3
1.3	Répartition de l'équipe	3
2	Architecture globale	4
2.1	Les cinq modules prévus	4
2.2	Récapitulatif des modèles utilisés	4
2.3	Structure du dépôt	5
3	Infrastructure et environnement	6
3.1	Migration du disque C vers le disque D	6
3.2	Cause réelle de l'espace disque C qui se remplissait	6
3.3	Matériel et limites de performance	6
4	Module reconnaissance vocale et détection de langue	7
4.1	Choix techniques	7
4.2	Amélioration : biais de vocabulaire	7
4.3	Protocole de test qualité	7
5	Module RAG (génération de réponses)	8
5.1	Architecture	8
5.2	Amélioration 1 : détection d'ambiguïté entre programmes	8
5.3	Amélioration 2 : qualité des données – fiches "gouvernance"	9
5.4	Amélioration 3 : extraction et regroupement du dossier Admission	9
5.5	Audit de la base de connaissances : fiches peu détaillées ou hors-sujet	9
5.6	Nettoyage : préfixe d'interface web et doublons de contenu	10
5.7	Clôture de l'audit : les 4 chantiers restants traités	10
5.8	Correction : cul-de-sac de la clarification en mode vocal	11
5.9	Nettoyage : catégorie "Vie étudiante"	12
5.10	Consolidation par sujet : facilités de paiement	12
5.11	Consolidation par sujet : réductions sur les frais de scolarité	12
5.12	Écart de vocabulaire : "conseil de classe" vs libellés calendrier	13
5.13	Refonte complète des calendriers : extraction groupée par type d'événement	13
5.14	Confirmation utilisateur : le calendrier semestre 2 est commun à tous les groupes	14
5.15	Règlements de scolarité : ré-extraction par article	15
5.16	Test qualité sur 35 questions et corrections	15
5.17	Contradictions entre sources sur les frais 2024-2025	16
5.18	Test qualité sur 100 questions et confirmation des montants par email officiel	17

6	Module synthèse vocale (TTS)	18
6.1	Choix techniques	18
6.2	Réglage de la vitesse	18
7	Backend / API REST	19
7.1	Endpoints disponibles	19
7.2	Ce qui reste à faire côté backend	19
8	Application d'ingestion de la base de connaissances (extraction_app)	20
8.1	Contexte et objectif	20
8.2	Pipelines d'extraction par type de source	20
8.3	Garde-fous obligatoires avant sauvegarde	21
8.4	Authentification et planificateur	21
8.5	Découverte : corruption d'encodage dans les fichiers .docx d'admission	21
8.6	Limites assumées	22
9	Tests et validation	23
9.1	Tests manuels effectués	23
9.2	Scripts de test disponibles	23
10	Limites connues et travail restant	24
10.1	Limites techniques actuelles	24
10.2	Travail restant	24

1. Introduction et contexte

1.1 Objectif du projet

Le projet **ESPRIT AI Receptionist** vise à concevoir un assistant vocal intelligent capable de répondre automatiquement aux appels téléphoniques des étudiants et parents de l'école ESPRIT, en français et en arabe tunisien (dialecte). L'assistant doit pouvoir comprendre une question posée à l'oral, y répondre à partir d'une base de connaissances officielle de l'école, et reformuler cette réponse à l'oral.

1.2 Contrainte fondamentale : 100% gratuit et local

L'ensemble des briques techniques utilisées sont **gratuites et fonctionnent entièrement en local** sur la machine de développement : aucune clé API payante, aucun service cloud facturé à l'usage. Ce choix structure toutes les décisions techniques du projet (choix des modèles, de l'infrastructure, des bibliothèques).

1.3 Répartition de l'équipe

- **Ikbel** : module de génération de réponses (RAG) – préparation de la base de connaissances (FAQ, web scraping à partir d'un mail, règlements, calendrier, admissions, paiements). Procédure de secours si l'information n'est pas disponible : contacter directement les services concernés (Admission : rez-de-chaussée bloc E ; FAQ : réseaux sociaux + réception entrée bloc E ; Calendrier : email sinon service scolarité ; Paiement : service financier, 1^{er} étage bloc E).
- **aziz** : backend / API REST.
- **Auteur de ce rapport** : reconnaissance vocale (STT), détection de langue, synthèse vocale (TTS), et backend / API REST.

À faire, non encore attribué explicitement : dashboard (interface CRUD documents + statistiques/indicateurs) et conteneurisation Docker + déploiement.

2. Architecture globale

2.1 Les cinq modules prévus



Module	Statut
Détection de langue (FR / arabe tunisien)	Fait – faster-whisper, 100% local
Reconnaissance vocale (STT)	Fait – faster-whisper (medium), testé
Compréhension de la question (LLM)	Fait – intégrée au module RAG
Génération de réponses (RAG)	Fait – testé et amélioré (voir chap. 5)
Synthèse vocale (TTS)	Fait – Piper, voix française
API REST (backend)	Fait – 5 endpoints fonctionnels
Dashboard (CRUD + stats/KPI)	Non commencé
Docker (conteneurisation + déploiement)	Non commencé

2.2 Récapitulatif des modèles utilisés

Étape	Modèle / outil	Détails
Détection de langue	faster-whisper (medium)	Même modèle que le STT (partagé en mémoire), CPU, quantification int8
Reconnaissance vocale (STT)	faster-whisper (medium)	100% local, device=cpu, compute_type=int8
Embeddings (recherche RAG)	sentence-transformers (paraphrase-multilingual-mpnet-base-v2)	Multilingue français + arabe, local
Base vectorielle (RAG)	ChromaDB	Stockage des embeddings, persisté dans vector_store/
Génération de réponse (RAG/LLM)	Ollama (qwen2.5:7b-instruct)	7 milliards de paramètres, 50%/50% CPU/GPU (VRAM limitée à 4 Go)

Étape	Modèle / outil	Détails
Synthèse vocale (TTS)	Piper (fr_FR-siwis-medium)	Français uniquement, 100% local

Tous ces modèles sont **gratuits, open-source, et tournent entièrement en local** (aucune clé API), conformément à la contrainte fondamentale du projet (voir section 1.2).

2.3 Structure du dépôt

data/	Collecte et preparation de la base de connaissances
raw_data/	Documents bruts (PDF, pages HTML scrapees)
processed/	Base de connaissances nettooyee (site_esprit_clean.json)
scripts/	Scripts de scraping, extraction PDF, nettoyage
modules/	
rag/	Generation de reponses (RAG)
language_detection/	Detection francais / arabe
speech_to_text/	Reconnaissance vocale
text_to_speech/	Synthese vocale
backend/	API REST reliant les modules
routers/	rag.py, stt.py, tts.py
extraction_app/	Appli separee : ingestion controlee de nouvelles fiches dans site_esprit.json (PDF/Excel/image/URL)
dashboard/	(a venir)
docker/	(a venir)
tests/	Scripts de test manuels
voice_models/	Modeles de voix Piper (gitignore, telechargeable)
vector_store/	Base vectorielle ChromaDB (gitignore, regeneree)

3. Infrastructure et environnement

3.1 Migration du disque C vers le disque D

Le projet a été déplacé du disque C: vers le disque D: (d:\stage_ia_receptionist) par manque d'espace sur C. Les caches de modèles ont été redirigés vers D via des variables d'environnement Windows persistantes :

- `OLLAMA_MODELS = D:\ai_cache\ollama_models\models`
- `HF_HOME = D:\ai_cache\huggingface` (cache Hugging Face : embeddings + Whisper)

Piège identifié : un terminal déjà ouvert avant la création de la variable d'environnement ne la voit pas (les variables User ne sont lues qu'au démarrage d'un nouveau process). Toujours ouvrir un terminal neuf, ou refaire `$env:HF_HOME = "D:\ai_cache\huggingface"` manuellement si un téléchargement inattendu se déclenche.

3.2 Cause réelle de l'espace disque C qui se remplissait

Contrairement à l'hypothèse initiale (les modèles IA), la vraie cause identifiée est double :

1. **Docker Desktop** : les disques virtuels WSL2 (`docker_data.vhdx` $\approx$ 8,7 Go et `ext4.vhdx` $\approx$ 3,5 Go) vivent sur C par défaut et ne rétrécissent jamais automatiquement. Solution proposée (pas encore appliquée) : déplacer l'emplacement du disque Docker vers D.
2. **Fichiers système Windows** : `pagefile.sys` ($\approx$ 19 Go) et `hiberfil.sys` ($\approx$ 6,3 Go), gonflés par la pression mémoire (16 Go de RAM, forte utilisation par Whisper + Ollama + Docker simultanément). Solution proposée : déplacer le fichier d'échange vers D via les paramètres de performance Windows (nécessite droits admin + redémarrage).

3.3 Matériel et limites de performance

- GPU : NVIDIA GeForce GTX 1650 Ti, **4 Go de VRAM seulement**.
- Ollama (modèle 7B) tourne en partage **50% CPU / 50% GPU** (le modèle ne tient pas entier dans la VRAM).
- Whisper (STT) tourne **entièrement en CPU**, décision volontaire pour ne pas entrer en conflit de VRAM avec Ollama (voir `modules/speech_to_text/config.py`).
- Conséquence : `/api/converse` (cycle complet voix→voix) prend environ **20 à 40 secondes** par question, une fois les modèles chargés en mémoire (le tout premier appel après un redémarrage du serveur est plus lent : chargement des modèles en RAM).

4. Module reconnaissance vocale et détection de langue

4.1 Choix techniques

Basé sur `faster-whisper`, modèle `medium`, 100% local et gratuit. Les modules `speech_to_text` et `language_detection` partagent le même modèle chargé en mémoire (`modules/speech_to_text/model.py`) pour éviter de le charger deux fois.

4.2 Amélioration : biais de vocabulaire

Un paramètre `INITIAL_PROMPT` a été ajouté (`modules/speech_to_text/config.py`) pour biaiser le décodage de Whisper vers le vocabulaire du domaine (noms propres ESPRIT, termes admissions/scolarité) et quelques tournures tunisiennes courantes (باش، فم، نجم، باش، Coût nul, réversible, aucune garantie universelle – aide sur des mots spécifiques, pas sur la grammaire ou l'accent en général).

Limite connue et documentée par la recherche : Whisper (même en version large) est entraîné très majoritairement sur de l'arabe standard et quelques dialectes bien dotés en données (égyptien...). Le tunisien (derja) reste sous-représenté. Des modèles communautaires dédiés existent (`sadeemar/whisper-finetuned-Tunisian`, `SalahZa/Tunisian_Automatic_Speech_Recognition`) mais sans garantie de qualité supérieure vérifiée, ou avec un changement d'architecture non négligeable (`WavLM/SpeechBrain`). Un fine-tuning maison demanderait de collecter et annoter nos propres enregistrements tunisiens – hors périmètre réaliste du stage.

4.3 Protocole de test qualité

Un script dédié (`tests/test_language_quality.py`) guide l'utilisateur à travers 8 sujets représentatifs de la base de connaissances (admission, frais, paiement, calendrier, stage, documents, contact général, hors-sujet), enregistre chaque réponse en arabe tunisien, l'enchaîne dans le pipeline complet, et sauvegarde un rapport JSON (`Bureau\test_arabe_resultats.json`) pour analyse structurée.

5. Module RAG (génération de réponses)

5.1 Architecture

- **Base vectorielle** : ChromaDB, persistée dans `vector_store/` (regénérée via `python -m modules.rag.ingest`).
- **Embeddings** : sentence-transformers, modèle paraphrase-multilingual-mpnet-base-v2 (multilingue FR/AR).
- **LLM** : Ollama, modèle qwen2.5:7b-instruct (bon support multilingue parmi les modèles open-source).
- **Seuil de pertinence** : `SCORE_THRESHOLD = 0.45` – en dessous, redirection vers un message de fallback (service humain).

5.2 Amélioration 1 : détection d’ambiguïté entre programmes

Problème constaté : une question vague (“*combien ça coûte pour étudier à ESPRIT ?*”) piochait dans la première fiche la mieux notée, sans vérifier qu’elle correspondait bien au bon programme parmi plusieurs (cours du jour, cours du soir, EMBA...). Exemple observé : réponse de 20 000 DT (tarif EMBA) au lieu des 8 000 DT du programme d’ingénieur généraliste.

Solution implémentée (`modules/rag/programmes.py` + `modules/rag/pipeline.py`) :

1. Chaque fiche est associée à un *programme* (cours du jour tunisiens / internationaux, cours du soir, EMBA, alternance...) déduit de son URL source.
2. Si, parmi les meilleures fiches retrouvées (à une marge de score près), au moins deux programmes différents sont représentés **et** que la question ne précise pas lequel, le système répond directement par une demande de clarification – **sans appeler le LLM** (quasi instantané, ~0,3s au lieu de 15 à 40s).
3. Garde-fou anti-faux-positif : la détection ne se déclenche que si le *meilleur* résultat est déjà spécifique à un programme (sinon, une question sur un service général – stage, carrière... – ne serait pas concernée).
4. Garde-fou supplémentaire : si la question mentionne déjà explicitement un programme par mot-clé (ex. “cours du soir”), pas de clarification demandée.

Résultat validé : questions précises (“frais des cours du soir”) répondent normalement ; questions vraiment vagues (“comment puis-je payer ?”) déclenchent la clarification ; questions sur des services généraux (“service de stage”) ne déclenchent plus de faux positif.

5.3 Amélioration 2 : qualité des données – fiches “gouvernance”

Problème constaté : une question sur le département des stages ne mentionnait jamais le contact réel (Majdi Gharbi) alors que l’information existait dans la base.

Cause racine : `modules/rag/ingest.py` embed le texte `"{titre} : {contenu}"`. Pour les 28 fiches de la page gouvernance, le titre était le **nom de la personne** (ex. “Majdi GHARBI”), qui dilue l’embedding au lieu de l’aider – un nom propre n’apporte aucun signal utile à la recherche sémantique.

Correction appliquée (`data/processed/site_esprit_clean.json`, 28 fiches) :

- **Titre** : nom de la personne → sujet/département (ex. “Département des stages”).
- **Contenu** : reformulé en phrase naturelle (ex. “*Majdi GHARBI est la personne à contacter pour toute question sur département des stages. Email : majdi.gharbi@esprit.tn.*”).
- Base réindexée.

Résultat mesuré : score de similarité passé de absent du top 10 à 0,601 (2^e position) sur la question “qui contacter pour le département des stages ?”. La réponse finale mentionne désormais correctement le contact.

Limite honnête : la correction fonctionne pour les formulations proches du vocabulaire de la fiche. Des formulations plus détournées (“je cherche un stage, qui dois-je contacter ?”) ne remontent toujours pas cette fiche – limite inhérente à la recherche sémantique sur des textes courts.

5.4 Amélioration 3 : extraction et regroupement du dossier Admission

Objectif : les documents `.docx` de `data/raw_data/Admission/` (FAQ par programme, conseillers admission) n’étaient pas encore dans la base de connaissances. Plutôt que d’indexer une fiche par question isolée, le contenu est **regroupé par section** (titres en gras du document, ex. “Admission et Inscription”, “Modalités Financières”) : ni trop court (une seule question, mal retrouvée en recherche sémantique – voir amélioration 2), ni trop long (dilution de l’embedding sur plusieurs sujets différents).

Script : `data/scripts/extract_admission_docx.py` – analyseur souple gérant plusieurs variantes de mise en forme (question/réponse en paragraphes séparés ou combinés, sections purement descriptives), avec nettoyage des notes d’auteur parasites (ex. “(mettre 3 boutons cliquables...)”). Chaque fiche est rattachée au bon programme (réutilisation des URLs canoniques déjà utilisées par le système de détection d’ambiguïté, section 5).

Résultat : 27 fiches ajoutées (785 fiches au total avant l’audit ci-dessous), comparées à `site_esprit_clean.json` existant (aucun doublon exact). Test sur une question vague (“comment ça marche l’inscription aux cours du soir ?”) : réponse complète en 5 étapes – bonne, mais pas systématiquement grâce aux nouvelles fiches précises (beaucoup de contenu existant fait concurrence en recherche).

5.5 Audit de la base de connaissances : fiches peu détaillées ou hors-sujet

Un audit systématique (fiches de moins de 60 caractères, 86 sur 785) a révélé plusieurs catégories de problèmes :

1. **Conflit de calendriers** (~60 fiches) : trois fichiers calendrier différents (1,2,3&4A / 5A / 5DS9&5Infini3) avec des dates différentes pour des événements similaires (ex. vacances d'hiver) – risque de réponse mélangeant les années/promotions. **Pas encore corrigé.**
2. **Bruit PDF hors-sujet** : le document RDI_CATALOG_2023-2024.pdf (rapport annuel de recherche académique – équipes de recherche, publications scientifiques, conférences) s'est révélé **entièrement hors périmètre** pour un assistant de réception admissions/scolarité. **Corrigé** : 49 fiches supprimées (785 → 736 fiches), base réindexée, aucune régression constatée (une des nouvelles fiches "Modalités Financières" est même remontée en 1^{re} position sur un test déjà validé, probablement grâce à moins de contenu parasite en concurrence).
3. **Fiches contact avec nom en titre** (ex. "Ramla Ben Ouirane", "Pr. Faouzi Kamoun") : même défaut que la correction "Département des stages" ci-dessus. **Pas encore corrigé.**
4. **Lignes de grille tarifaire isolées** (ex. "550DT" / "Demi-pension (F1)...") : extraites une par une sans le contexte du tableau d'origine. Lié au sous-dossier Admission/Grilles tarifaires/ (PDF, Excel, zip), **pas encore traité.**

5.6 Nettoyage : préfixe d'interface web et doublons de contenu

Deux problèmes supplémentaires, découverts en creusant le mécanisme même de la recherche sémantique (comment le RAG choisit une fiche pour une question donnée) :

1. **Préfixe d'interface parasite** : 85 fiches sur 736 avaient un titre commençant par "*Ouvrir la visibilité du contenu* : " – le libellé du bouton "déplier" d'un accordéon FAQ, scrapé par erreur avec la vraie question. Comme `modules/rag/ingest.py` donne du poids au titre dans l'embedding, ce préfixe identique sur 85 fiches diluait leur pertinence. **Corrigé** : préfixe retiré (regex simple, aucun risque – ne touche jamais le contenu factuel).

2. **Doublons de contenu préexistants, révélés par le nettoyage** : une fois les titres nettoyés, ces fiches ont mieux remonté en recherche – révélant que **4 groupes de fiches quasi-identiques** existaient déjà entre plusieurs pages de programmes (ex. "Les diplômes délivrés par ESPRIT sont-ils reconnus ?" dupliqué mot pour mot entre les pages "cours-du-jour-tunisiens" et "cours-du-jour-internationaux"). Conséquence concrète : le système de détection d'ambiguïté (section 5) demandait à tort de préciser le programme, alors que la réponse est identique quel que soit le programme choisi. **Corrigé** : les 4 groupes fusionnés en une seule fiche chacun, retaguée avec une source générique non rattachée à un programme spécifique (736 → 730 fiches). Testé et validé : la question "le diplôme ESPRIT est-il reconnu ?" répond désormais directement, sans fausse demande de clarification.

Cet épisode illustre bien la nature itérative du nettoyage de données pour un RAG : corriger un problème (préfixe parasite) peut révéler un autre problème resté invisible jusque-là (doublons), simplement parce que les fiches concernées remontent différemment en recherche une fois nettoyées.

5.7 Clôture de l'audit : les 4 chantiers restants traités

1. **Fiches contact restantes** : 2 dernières fiches avec un nom de personne en titre ("Ramla Ben Ouirane", "Pr. Faouzi Kamoun") corrigées selon le même schéma que "Département des stages". La fiche du centre de carrière a ensuite été reformulée une seconde fois (titre sous forme de question "Qui contacter au centre de carrière ?") après un test montrant qu'elle perdait face à des FAQ génériques "qui contacter" d'autres programmes.

2. Conflit des calendriers : les 67 fiches issues des 3 fichiers calendrier (1,2,3&4A / 5A / 5DS9&Infini3) sont désormais préfixées avec leur promotion (ex. “*Pour 5ème année (5A) – le mardi 2025-12-30 : Vacances d’hiver.*”) : même si plusieurs dates concurrentes remontent en recherche, la réponse ne peut plus mélanger les promotions.

3. Grilles tarifaires (Admission/Grilles tarifaires/) : 7 PDF + 1 Excel extraits avec pdfplumber en mode *tableau* (pas texte brut, pour ne pas mélanger les colonnes). Le fichier zip du dossier s’est révélé être une copie exacte de 4 PDF déjà présents (tailles de fichier identiques) – ignoré. 10 fiches ajoutées : foyer et restauration (tunisiens/internationaux), historique des frais 2018-2025 (cours du jour et cours du soir), classe préparatoire (PREPA, nouveau programme ajouté au système de détection d’ambiguïté), et ESPRIT School of Business (Bachelor/Master). Chaque montant a été vérifié manuellement contre le tableau source avant d’être transcrit en phrase complète – aucune réécriture automatique de chiffres.

4. Champs source et type : vérifié dans le code que ni l’un ni l’autre n’est jamais envoyé au LLM ni à l’embedding (ingest.py et generator.py n’utilisent que titre + contenu). type était réellement mort code (jamais relu) – supprimé de toutes les fiches et des 6 scripts d’extraction. source est en revanche activement utilisé par la détection d’ambiguïté et les citations – conservé. Le vrai risque de confusion identifié était ailleurs : **203 fiches** avaient un titre brut de type “*nom-fichier.pdf - page N*” (ex. Reglement-scolarité-24-25-cs.pdf - page 8), qui pollue l’embedding (titre pesé dans la recherche). Un test empirique a confirmé qu’un titre nettoyé (ex. “*Règlement de la scolarité - Cours du soir 2024-2025*”) score mieux (0,667) qu’un titre brut (0,659) et qu’une absence de titre (0,654) – les 203 titres ont été remappés vers des noms lisibles pour les 18 documents concernés.

5. Robustesse de la détection d’ambiguïté : un cas limite a révélé que la règle “ne se déclenche que si le meilleur résultat a un label” était fragile – un écart de score de 0,001 entre deux fiches (bruit d’embedding) suffisait à faire basculer une question générale (“comment fonctionne le service de stage ?”) en fausse alerte. **Corrigé** : la règle regarde maintenant un consensus sur les 3 meilleurs résultats (majorité avec un label de programme) plutôt que le seul premier résultat – plus robuste au bruit, sans perdre la détection des vrais cas ambigus.

Base de connaissances finale : **740 fiches** (730 avant ce chantier, +10 fiches de grilles tarifaires). Régression complète validée sur 5 scénarios de référence (précis / vague / général / contact / calendrier) avant clôture de ce chantier.

5.8 Correction : cul-de-sac de la clarification en mode vocal

Problème découvert en testant /api/converse avec une vraie voix : sur une question vague (“comment puis-je payer ?”), l’appelant recevait la demande de clarification (“précisez le programme”) *en audio*, sans aucun moyen d’y répondre – /api/converse est un aller-retour unique (audio → audio), sans mémoire de conversation entre deux appels. Demander une précision qu’on ne peut pas recevoir est un cul-de-sac inacceptable en usage téléphonique réel.

Correction : answer_question() accepte désormais un paramètre allow_clarification (modules/rag/pipeline.py). Quand il vaut False, le LLM génère toujours une réponse directe même en cas d’ambiguïté détectée – une note est ajoutée au prompt (modules/rag/generator.py) lui demandant de couvrir les principaux programmes concernés plutôt que de refuser de répondre.

Après test, décision prise d’appliquer ce comportement **uniformément aux 3 endpoints** (/api/ask, /api/voice-ask, /api/converse) : plus aucune demande de clarification nulle part, pour une expérience cohérente quel que soit le canal.

Testé : la même question ambiguë donne désormais une réponse directe et utile sur les 3 endpoints (ex. détail des tranches de paiement pour plusieurs programmes, avec une invitation à vérifier les spécificités de son propre programme), au lieu d’une question sans issue.

5.9 Nettoyage : catégorie “Vie étudiante”

Problème signalé : la catégorie “Vie étudiante” (144 fiches) contenait beaucoup de détail et plusieurs fiches jugées inutiles. Analyse : 103 des 144 fiches étaient en réalité des **règlements de scolarité mal catégorisés** (laissés intacts, car potentiellement utiles pour de vraies questions réglementaires) ; les 41 fiches restantes (clubs, alumni, étudiants internationaux, cellule d’écoute) constituaient la vraie “vie étudiante”, ciblée par ce nettoyage.

Supprimé (9 fiches, aucune valeur informative) :

- 6 fiches “Visite virtuelle” : légendes marketing génériques sans même le nom de la salle décrite (ex. “un espace calme et inspirant”).
- 2 fiches “Règlement intérieur” : renvoyaient simplement à “télécharger le règlement ailleurs”, sans contenu réel.
- 1 fiche fusionnée avec une autre (cellule d’écoute, partie 2/2 courte).

Raccourci (32 fiches : 20 clubs, 3 alumni, 6 étudiants internationaux, 3 cellule d’écoute) : style promotionnel/verbeux retiré, tous les faits conservés (emails, téléphones, adresse, dates de fondation). Médiane de longueur : 821 → 167 caractères.

Erreur de données trouvée et corrigée au passage : la fiche “Cellule d’écoute” contenait une phrase sur la communauté Alumni, copiée par erreur depuis une autre page lors du scraping initial – retirée car factuellement fausse dans ce contexte.

Testé : question sur un club (robotique) → bon club identifié avec email (score 0,722) ; question de détresse psychologique (formulation naturelle) → cellule d’écoute remontée en premier avec coordonnées complètes.

5.10 Consolidation par sujet : facilités de paiement

Demande : quand la question porte sur un sujet transversal (ex. “puis-je payer par facilité ?”), la réponse doit couvrir toutes les méthodes existantes, pas une seule au hasard parmi des doublons.

Constat : le paragraphe “Financement” (prêt Fondation ESPRIT, banques partenaires Amen/Zitouna/UBCI, jusqu’à 36 mensualités) était dupliqué quasi à l’identique sur **6 fiches différentes** (plusieurs pages de grilles tarifaires, années 2024-2025 et 2025-2026). Avec seulement 5 fiches remontées par requête (TOP_K), 4 des 5 places pouvaient être occupées par la même information répétée, au détriment d’informations réellement différentes (ex. le financement spécifique à l’EMBA : 2 à 5 tranches, partenariat Banque Zitouna).

Corrigé : les 6 doublons remplacés par une fiche unique “Facilités de paiement et financement (tous programmes)”, couvrant le cas général (cours du jour/soir) et le cas EMBA côte à côte.

5.11 Consolidation par sujet : réductions sur les frais de scolarité

Constat : l’information sur les réductions (mérite académique, fratrie, cours du soir) était éparpillée sur une douzaine de fiches (FAQ générale, pages Admissions spécifiques par programme, fiches Paiements 2024-2025), avec des taux légèrement différents selon la fiche (25%/10% ici, 30%/15% là) sans qu’il soit évident qu’il s’agissait en réalité de la même politique exprimée sous forme de fourchettes (“excellence haute” 25–30%, “excellence standard” 10–15%) plutôt que de valeurs contradictoires. Aucune fiche ne couvrait le cas des étudiants internationaux

(plafonds 30%/40%), la règle de non-cumul, ni la procédure de demande (dossier conjoint signé par les tuteurs + copies des cartes d'identité).

Corrigé : ajout d'une fiche unique consolidée (`paiements_reductions_scolarite_detail_complet`) couvrant les quatre volets (mérite, fratrie, cours du soir, conditions générales/démarche) à partir d'informations internes confirmées. Les fiches existantes (issues du site, déjà validées) n'ont pas été supprimées – elles restent cohérentes avec la nouvelle fiche (leurs taux sont des cas particuliers dans les fourchettes données), seulement incomplètes prises isolément.

Test de bout en bout après ré-indexation (`python -m modules.rag.ingest`) : la nouvelle fiche remonte en premier (*score* = 0.876) sur la question “Quelles sont les réductions possibles sur les frais de scolarité ?”. La réponse générée reste volontairement concise (persona vocal, SYSTEM_PROMPT impose des “réponses courtes et naturelles à l'oral”) et ne récite donc pas exhaustivement tous les pourcentages – comportement voulu pour un appel téléphonique, mais qui mérite d'être validé avec l'utilisateur si le besoin réel est une réponse plus détaillée pour ce type de question transversale.

5.12 Écart de vocabulaire : “conseil de classe” vs libellés calendrier

Constat : la question “Quelle est la date des conseils de classe ?” ne remontait **aucune** des fiches calendrier pertinentes (meilleur score 0,69, sur une fiche totalement hors-sujet), alors que la question “Quelle est la date du conseil de la session principale ?” fonctionnait (score 0,765). Cause : les 4 fiches calendrier concernées utilisent exclusivement les libellés bruts du PDF source (“Conseils Session Principale”, “Conseils de la session principale”) et ne contiennent jamais le mot “classe” – le terme générique que les appelants utilisent naturellement n'existe nulle part dans le texte embeddé de ces fiches.

En vérifiant les PDF sources (`pdfplumber`, avant toute correction) : le libellé “Conseils de la session principale” apparaît bien deux fois à l'identique dans `Calendrier_5DS9 & 5INFINI3.pdf` (16 et 30 janvier) – confirmé fidèle au PDF, pas une erreur d'extraction, donc non modifié. En revanche, la coquille “Session de rattraoage” (pour “rattrapage”) est également confirmée présente telle quelle dans le PDF source (`Calendrier_5A_2526_S1.pdf`) – corrigée dans la base car elle nuit à la recherche sans changer le sens.

Corrigé : les 4 fiches calendrier mentionnant un conseil (2 pour 5ème année/5A, 2 pour 5DS9/5Infini3) enrichies pour inclure explicitement “conseil de classe” en titre et contenu, en plus du libellé original conservé.

Test après ré-indexation : la question “conseils de classe” fait maintenant remonter les deux fiches 5A (scores 0,679 et 0,635) et la réponse générée donne correctement les deux dates (3 décembre : session principale, 24 décembre : rattrapage). Point de vigilance : les scores restent inférieurs à certains résultats hors-sujet (0,69/0,68) – la correction fonctionne mais reste proche du seuil, une base plus large de synonymes (ex. mentionner aussi “conseil de classe” dans les fiches 1^{re} à 4^e année si un tel évènement existe dans leur calendrier) resterait à vérifier si le problème réapparaît sur d'autres formulations.

5.13 Refonte complète des calendriers : extraction groupée par type d'événement

Demande : l'ancienne extraction (`data/scripts/extraction_calendrier.py`) produisait une fiche minuscule par événement individuel (une par jour), jugée insatisfaisante. Un nouveau

fichier a par ailleurs été ajouté (Calendrier du deuxième semestre_2526.pdf), à intégrer également.

Nouveau script : data/scripts/reextract_calendrier_groupe.py. Reprend la reconstruction de date fiable (extraction de tableaux pdfplumber, déjà éprouvée), mais regroupe les événements par **type** (rentrée/débuts de semestre, APP0, examens session principale, examens session de rattrapage, conseils de classe, proclamation des résultats, fin des cours, vacances, soutenances, autres) plutôt qu'un par jour – une seule fiche cohérente par (groupe d'année × type) listant toutes les dates correspondantes.

Bug d'extraction découvert et corrigé : en inspectant la structure brute des tableaux pdfplumber (pas seulement le texte à plat), certains événements dont le libellé est trop long pour une seule case (ex. "Examens de la Session de Rattrapage") débordent visuellement sur la case du jour **suivant** dans le PDF source – pdfplumber attribue alors à tort le second morceau de texte ("Rattrapage" seul) à ce jour suivant, créant deux événements fantômes tronqués au lieu d'un seul événement complet correctement daté. Corrigé par une fusion des fragments de continuation reconnus (CONTINUATION_FRAGMENTS) avec l'événement de la veille, quand les deux jours sont consécutifs. Egalement filtrés : les numéros de semaine ("S1" à "S16") affichés dans la grille, qui ne sont pas de vrais événements.

Les 45 anciennes fiches "Calendrier*" supprimées de site_esprit_clean.json et site_esprit.json, remplacées par 34 fiches regroupées (ajoutées en fin de fichier). Après ré-indexation, trois questions testées avec de bons scores : date de l'APP0 (0,783), début des études 5ème année (0,751), conseils de classe (0,771 – contre 0,679 avec la version précédente, et 0,69 sur une fiche hors-sujet avant tout correctif). La réponse sur les conseils de classe mentionne aussi correctement l'absence de date pour les années 1^{re} à 4^e (vérifié : ce calendrier ne contient effectivement aucun événement "conseil").

5.14 Confirmation utilisateur : le calendrier semestre 2 est commun à tous les groupes

Point laissé ouvert plus haut : confirmé par l'utilisateur que Calendrier du deuxième semestre_2526.pdf s'applique de la même façon aux 5 groupes d'année (1^{re} à 5^e), malgré l'absence de qualificatif dans son titre. Les événements de ce fichier sont donc désormais fusionnés dans les fiches de **chacun** des 3 groupes existants (au lieu d'une catégorie à part), avec la source combinée des deux fichiers d'origine par fiche concernée.

Bug de génération découvert en aval : même après cette fusion, la question "Quand commence la session de contrôle pour les 1ère année ?" produisait toujours une réponse vague et non sourcée ("généralement en février ou mars... vérifiez le site web"), alors que la fiche pertinente était bel et bien récupérée (2^e sur 5 sources, score 0,692) – le LLM ne faisait simplement pas le lien entre "session de contrôle" (terme de l'utilisateur) et "session de rattrapage" (terme utilisé dans la fiche), et ignorait cette source dans sa réponse malgré sa présence dans le contexte fourni. Corrigé par le même type de synonyme explicite que pour "conseil de classe" : les fiches de type rattrapage portent désormais aussi la mention "(session de contrôle)".

Après ce correctif et une nouvelle ré-indexation : la fiche concernée remonte en **premier** résultat (score 0,706, contre 2^e position à 0,692 avant), et la réponse générée cite désormais une vraie date de la base (“la session de contrôle commencera le mercredi 24 juin 2026”) au lieu d’une réponse générique inventée.

5.15 Règlements de scolarité : ré-extraction par article

Constat le plus important de cette session : les 103 fiches de règlement (cours du jour, cours du soir, alternance, Private School en anglais) étaient en réalité des **pages brutes de PDF** (une fiche = une page, 100 à 2000 caractères), et non des fiches organisées par sujet – une page pouvant couper un article en plein milieu ou mélanger la fin d’un article et le début du suivant. Cause : ces 4 documents avaient été traités par le scraper web générique (extraction page par page), qui ignore la structure “Article N : Titre” pourtant bien présente dans les PDF sources.

Correction : nouveau script `data/scripts/reextract_reglements.py`, qui réextrait les 4 documents en respectant leur structure par article :

- Découpage sur le motif `Article N :`, suppression du sommaire (table des matières) et des lignes de sommaire à pointillés (ASCII . ou ellipses unicode ...) qui polluaient certains articles.
- Les articles trop longs (ex. “Régime des examens”, “Charte de l’étudiant”, 8 000 à 13 700 caractères – ils regroupent des sous-clauses non numérotées) sont sous-découpés en parties de 2000 caractères maximum, comme pour les autres documents du projet.
- Dédoublonnage des titres d’article répétés par erreur dans le PDF source.

Doublon supplémentaire découvert et supprimé : 23 fiches “cours du jour” existaient déjà, issues d’une extraction antérieure non documentée (source = nom de fichier local, sans URL) – contenu identique à la nouvelle extraction, supprimées au profit de la version cohérente avec le reste de la base (source en URL `https://`).

Résultat : 103 fiches fragmentées → **88 fiches cohérentes par article et par parcours**. Base de connaissances finale : **685 fiches**. Testé : question sur les conditions d’obtention du diplôme (cours du jour) → score de pertinence 0,782 (contre absence de structure claire avant), réponse complète et exacte citant l’article source.

5.16 Test qualité sur 35 questions et corrections

Un test systématique sur 35 questions couvrant 6 catégories (paiements, calendrier, règlements, admissions, vie étudiante, questions pièges) a été mené via `answer_question()` directement (script `_test_batch_questions.py`, temporaire). Deux problèmes réels ont été détectés et corrigés :

1. Mauvaise identité du directeur. À la question “quel est le nom du directeur actuel d’ESPRIT ?”, le modèle répondait *Lamjed Bettaieb (Directeur Général Adjoint)* au lieu du vrai Président-Directeur Général, *Tahar Ben Lakhdar*. Cause : 9 fiches de gouvernance différentes (“Directeur administratif et financier”, “Directeur des études”, “Directeur Général Adjoint”...) contiennent toutes le mot générique “Directeur” et scorent de façon quasi identique (0,475 à 0,561) – le LLM, recevant les 5 meilleures sans distinction claire de hiérarchie, ne priorisait pas le Président-Directeur Général. **Corrigé** : ajout d’une fiche dédiée “Qui est le directeur de l’école ESPRIT ?” répondant sans ambiguïté – score de similarité 0,656 (nettement au-dessus des 9 fiches concurrentes), résout le cas.

2. Réponse totalement corrompue (bug sévère). À la question “combien de fois je peux redoubler ?”, le modèle générait un texte **en chinois**, sans rapport avec la question (une conversation inventée sur la météo). Cause : le score de la meilleure fiche récupérée n'était que 0,451 (à peine au-dessus du seuil de fallback 0,45), noyé parmi des fragments de prix totalement hors-sujet (“20 000 DT TTC”, “3 200DT”...) – le vrai fait (“*le redoublement n'est toléré qu'une seule fois par cycle de formation*”) existait bien dans la base mais enfoui dans un article de règlement de 1856 caractères, mal capté par la recherche sémantique. **Corrigé** : ajout d'une fiche dédiée consolidant les 3 parcours, avec une phrase de désambiguïsation explicite (“un seul redoublement au total est autorisé, jamais deux”) après qu'une première version ait fait dire au LLM “deux redoublements” par mauvaise lecture d'une formulation par-parcours. Score final : 0,536 (validé empiriquement avant mise en production, comme pour les autres corrections de ce rapport).

Leçon retenue : un score de similarité proche du seuil de fallback (0,45) est un signal fiable de risque de génération de mauvaise qualité, y compris des dérives complètes (langue incorrecte, hors-sujet) – pas seulement des réponses imprécises. À surveiller pour d'éventuels ajustements futurs du seuil ou de garde-fous sur la génération.

5.17 Contradictions entre sources sur les frais 2024-2025

Un script de nettoyage externe (non écrit dans cette session) proposait de supprimer les fiches issues d'un email de paiement, jugées “fausses” par rapport à la grille tarifaire du site. Vérification avant exécution – deux cas distincts :

Cas 1 (résolu par suppression) : 4 fiches (admissions_cours_du_jour_tunisiens_003 à 006, valeurs de 2ème tranche par année) ne correspondaient pas exactement à la fiche historique sourcée depuis le PDF officiel de grille tarifaire (3 des 4 valeurs différaient de 50 à 100 DT). Contrairement à ce qu'affirmait le script, ce n'étaient pas des doublons identiques – mais le PDF officiel étant plus fiable qu'une page web potentiellement obsolète, les 4 fiches ont été supprimées.

Cas 2 (résolu par clarification, pas suppression) : le taux de réduction pour mérite académique différait entre l'email (30%/15%) et le site (25%/10%). Lecture attentive du contenu du site : celui-ci précise explicitement que le taux 25%/10% est *nouveau et applicable aux frais de 2025-2026*, décidé pendant l'année 2024-2025. Le taux de l'email est donc probablement correct pour 2024-2025, pas une erreur. Les deux valeurs ont été **conservées**, avec les titres clarifiés par année plutôt que l'une supprimée au profit de l'autre.

Cas 3 (non tranché, signalé pour vérification humaine) : le montant total des frais 2024-2025 diffère aussi (8200 DT selon l'email vs 8400 TND selon le site et une FAQ docx). Contrairement au cas 2, **aucune des fiches ne mentionne de réserve d'année** – les deux affirment ce montant pour la même année 2024/2025 sans nuance. Faute de moyen de vérifier laquelle est correcte, **aucune fiche n'a été supprimée ni fusionnée** : les deux valeurs restent dans `site_esprit_clean.json`, et le conflit est documenté séparément dans `data/processed/conflits_a_verifier` pour vérification manuelle auprès du service financier.

Principe appliqué : ne jamais trancher automatiquement une contradiction de données factuelles (montants, taux) sans preuve claire de laquelle est correcte – soit clarifier par un facteur objectif retrouvé dans le contenu (ici, l'année d'application), soit signaler explicitement pour vérification humaine plutôt que de deviner.

5.18 Test qualité sur 100 questions et confirmation des montants par email officiel

Un second test systématique, sur 100 questions couvrant 10 catégories, a révélé que le bug de dérive complète du LLM (section 5.16) n'était pas propre à la question du redoublement : il est réapparu sur une question sans rapport (classe préparatoire), confirmant qu'il s'agit d'un risque systémique lié à un score de pertinence faible, pas d'un cas isolé. Deux autres réponses se sont révélées fausses ou contradictoires (campus de Monastir, existence d'ESPRIT School of Business), à recouper avec les catégories internes de programmes.

L'utilisateur a ensuite fourni les deux emails officiels de réinscription (2024-2025 et 2025-2026) envoyés par la Direction, résolvant définitivement le conflit du montant total signalé plus haut : **8200 DT pour 2024-2025 est confirmé correct** (les fiches affirmant 8400 TND, aujourd'hui corrigées, étaient fausses) ; **8350 TND pour 2025-2026** est ajouté comme nouvelle donnée confirmée (4050 TND avant le 31/08/2025, 4300 TND avant le 15/01/2026), avec 7 moyens de paiement détaillés (en ligne, Amen Bank, BIAT, chèque, sur place, prélèvement 10 mensualités, traite avalisée).

Difficulté technique rencontrée et résolue : les nouvelles fiches 2025-2026, formulées de façon neutre, scoraient moins bien (0,68) que l'ancienne fiche 2024-2025 (0,85) pour une question portant explicitement sur 2025/2026 – confirmation que les embeddings distinguent mal les années pures, un chiffre à 4 chiffres se ressemblant sémantiquement d'une année à l'autre. Corrigé en répétant l'année plusieurs fois dans le contenu et en contrastant explicitement avec l'année précédente ("concerne spécifiquement 2025/2026, différent de 2024/2025") : score porté à 0,88, redevenant la source la plus pertinente. Le `conflits_a_verifier.json` a été mis à jour pour marquer ce conflit comme résolu.

6. Module synthèse vocale (TTS)

6.1 Choix techniques

Basé sur **Piper** (moteur de synthèse neuronale, 100% local, gratuit, aucune clé API). Voix française `fr_FR-siwis-medium` installée dans `voice_models/` (gitignored, retéléchargeable).

6.2 Réglage de la vitesse

La voix par défaut parlait trop vite pour un usage téléphonique. Un paramètre `SPEECH_LENGTH_SCALE = 1.25` (25% plus lent) a été ajouté dans `modules/text_to_speech/config.py`, appliqué via `SynthesisConfig` de Piper.

Limite connue : aucune voix arabe tunisienne (dialecte) disponible chez Piper – seulement de l'arabe standard moderne. Le module reste donc pertinent uniquement pour le français pour l'instant.

7. Backend / API REST

7.1 Endpoints disponibles

Endpoint	Description
GET /api/health	Vérifie que l'API répond.
POST /api/ask	Pose une question texte au RAG. Renvoie réponse + sources + indicateurs (used_fallback, needs_clarification).
POST /api/transcribe	Envoie un fichier audio, renvoie langue détectée + transcription.
POST /api/voice-ask	Envoie un fichier audio contenant une question orale, renvoie la réponse générée par le RAG (texte).
POST /api/speak	Envoie un texte, renvoie l'audio (.wav) synthétisé.
POST /api/converse	Cycle complet : audio (question) → transcription → RAG → audio (réponse).

7.2 Ce qui reste à faire côté backend

- Gestion d'erreurs explicite (fichier audio corrompu, texte vide, format non supporté).
- Tests automatisés (actuellement : validation manuelle via `curl` et `/docs`).
- Endpoints pour le futur dashboard (CRUD documents, statistiques).

8. Application d'ingestion de la base de connaissances (extraction_app)

8.1 Contexte et objectif

Le pipeline de collecte existant (`data/scripts/`) est un ensemble de scripts lancés manuellement, un par source, sans filtrage de pertinence, sans détection de doublon et sans journal d'activité. Pour permettre d'ajouter de nouvelles fiches à la base de connaissances de façon plus contrôlée – et sans risquer de polluer `site_esprit_clean.json` avec du contenu hors-sujet ou déjà présent – une application web séparée a été développée : `extraction_app/`. Elle vit dans un dossier indépendant du reste du projet (ne touche ni `backend/`, ni `modules/`, ni `data/scripts/`) et tourne sur un port distinct (8001) pendant que le backend principal reste sur le port 8000.

8.2 Pipelines d'extraction par type de source

Source	Logique
PDF	<code>pdfplumber</code> , découpage par Article N : si la structure existe (réutilise le même algorithme que <code>data/scripts/reextract_reglements.py</code> – retrait du sommaire, découpage des articles trop longs), sinon repli sur une fiche par page.
Word (.docx)	<code>python-docx</code> , regroupement par section (dernier titre en gras rencontré), détection des paires question/réponse à l'intérieur d'une section (réutilise la logique de <code>data/scripts/extract_admission_docx.py</code>) – une fiche par sujet complet plutôt que par mini-question isolée.
Excel (.xlsx)	<code>pandas/openpyxl</code> , une fiche par ligne pertinente de chaque feuille (concaténation colonne: valeur).
Image (.jpg/.png)	OCR <code>pytesseract</code> , puis même découpage que le PDF. Si le texte reconnu est trop court ou contient trop de caractères non reconnus, la fiche n'est jamais fusionnée silencieusement – elle est mise de côté dans <code>data/a_verifier.json</code> pour vérification manuelle.
URL	<code>requests</code> + <code>BeautifulSoup</code> , structure FAQ-accordéon dédiée pour <code>esprit.tn</code> (réutilise la logique de <code>data/scripts/web_scraping_espace_etudiant.py</code> , y compris le nettoyage des libellés d'accordéon), extraction générique par sections <code>h2/h3</code> sinon.

8.3 Garde-fous obligatoires avant sauvegarde

Deux vérifications s'appliquent à toute fiche candidate, quelle que soit sa source, avant toute fusion dans la base :

1. **Filtrage de pertinence** (`services/relevance_filter.py`) – rejette une fiche si ni son titre ni son contenu ne contiennent de mot-clé d'un des domaines couverts par l'assistant (admissions, scolarité, règlements, examens, calendrier, paiements, vie étudiante, stages, programmes, contacts).
2. **Vérification sémantique** (`services/semantic_dedup.py`) – TF-IDF + similarité cosinus (scikit-learn), comparée contre le contenu de **toutes** les fiches déjà présentes dans `site_esprit.json` (pas seulement même catégorie/source). Seuil configurable en haut de `config.py` (`SIMILARITY_THRESHOLD`).

Une dédup complémentaire par id empêche par ailleurs tout écrasement d'une fiche existante d'une autre source. Chaque extraction (manuelle ou planifiée) est journalisée dans `data/historique.json` : date, source, type, nombre de fiches ajoutées/rejetées (avec le titre de la fiche recoupée en cas de doublon), erreurs éventuelles – consultable sur la page `/history`.

8.4 Authentification et planificateur

Un compte unique protège l'interface (mot de passe généré aléatoirement au premier lancement, affiché une seule fois en console, jamais versionné). Un job APScheduler quotidien (03h00) rejoue automatiquement l'extraction pour les sources de type URL déjà soumises via l'interface – jamais pour les PDF/Excel/image uploadés manuellement, dont le fichier d'origine n'est pas conservé.

Point d'attention : `extraction_app` écrit uniquement dans `data/processed/site_esprit.json` (le fichier brut), jamais dans `site_esprit_clean.json` (celui que le RAG ingère réellement). Après une session d'extraction, il faut donc relancer manuellement `data/scripts/clean_knowledgebase` puis `python -m modules.rag.ingest` pour que les nouvelles fiches soient prises en compte par le RAG. Décision délibérée pour ne jamais court-circuiter le nettoyage exact-match existant.

8.5 Découverte : corruption d'encodage dans les fichiers .docx d'admission

En testant le nouveau pipeline Word sur `FAQ admission cours du soir.docx`, une section absente de la base (« Modalités Financières et Annulations ») a été correctement identifiée comme nouvelle par la vérification sémantique – mais son contenu s'est révélé truffé du caractère de remplacement Unicode U+FFFD () à la place de tous les accents français (« *Régler mes frais de scolarité* »). Vérification faite en lisant le .docx source directement avec `python-docx` (sans passer par `extraction_app`) : la corruption est bien dans le fichier source lui-même, pas introduite par le nouveau pipeline. Les 4 sections de ce même document déjà fusionnées en base via `data/scripts/extract_admission_docx.py` lors d'un travail antérieur présentent exactement la même corruption – ce n'est donc pas une régression, mais un défaut préexistant qui touche déjà des fiches en production.

U+FFFD signale une perte de données irréversible (un octet n'a pas pu être décodé à un moment donné et a été remplacé) – aucune astuce de réencodage ne peut récupérer les caractères d'origine. La correction nécessite d'obtenir une copie propre (non corrompue) des fichiers .docx concernés, pas un correctif côté code. Fiches concernées à ré-extraire une fois une copie saine disponible : les 4 sections déjà en base (admission_docx_faq_admission_cours_du_soir_01/02/03/05) et la section manquante détectée (04), et potentiellement d'autres fichiers du dossier data/raw_data/Admission/ non encore vérifiés un par un.

8.6 Limites assumées

- Le filtrage de pertinence est une simple correspondance de mots-clés, pas une compréhension du sens – peut laisser passer du hors-sujet ou rejeter du contenu pertinent mal formulé.
- La vérification sémantique (TF-IDF/cosinus) est lexicale, pas une vraie compréhension sémantique comme les embeddings du RAG – deux fiches formulées très différemment peuvent passer sous le seuil et être dupliquées.
- Le scraping générique ne peut ni exécuter de JavaScript ni contourner un blocage anti-bot ; si le contenu récupéré est vide ou anormalement court, aucune fiche trompeuse n'est sauvegardée – un avertissement explicite est affiché à la place.
- La dédup sémantique recalcule le vectoriseur TF-IDF sur toute la base à chaque extraction – viable à l'échelle actuelle (~600 fiches), mais à surveiller si la base grossit à plusieurs milliers de fiches (piste d'optimisation non implémentée : vecteurs pré-calculés).

Validation effectuée : les cinq pipelines testés de bout en bout via l'interface – Excel (fiche pertinente acceptée, fiche hors-sujet rejetée), PDF (ré-extraction d'un règlement déjà en base : 23 sections correctement détectées comme doublons, zéro ajout), Word (ré-extraction d'un FAQ admission déjà partiellement en base : 4 sections détectées comme doublons, 1 section manquante correctement identifiée comme nouvelle – voir découverte de corruption d'encodage ci-dessus), URL (page esprit.tn et site générique hors-sujet), et image (échec propre et message clair en l'absence du binaire Tesseract, sans plantage). Dédup par id vérifiée sur un ré-upload exact du même fichier. Job planifié déclenché manuellement sans erreur. site_esprit_clean.json confirmé non modifié par l'application.

9. Tests et validation

9.1 Tests manuels effectués

- `/api/health`, `/api/ask`, `/api/transcribe`, `/api/voice-ask`, `/api/speak`, `/api/converse` : tous testés de bout en bout avec succès.
- Enregistrements vocaux réels (français et arabe tunisien) via `tests/record_voice.py` puis upload manuel dans `/docs`.
- Cas limites du RAG testés : questions précises, questions vagues, questions sur des services généraux – comportement validé dans les trois cas après corrections.

9.2 Scripts de test disponibles

- `tests/test_rag_pipeline.py` – test interactif du RAG seul.
- `tests/test_stt_pipeline.py` – test interactif micro → transcription.
- `tests/record_voice.py` – enregistrement seul (sans transcription), pour tester ensuite via l'API.
- `tests/test_language_quality.py` – protocole structuré de test qualité arabe tunisien vs français (8 scénarios, rapport JSON sauvegardé).

10. Limites connues et travail restant

10.1 Limites techniques actuelles

- **Vitesse** : ~20 à 40 secondes par réponse vocale complète, due au GPU limité (4 Go VRAM) partagé entre Whisper (CPU) et Ollama (50% CPU / 50% GPU).
- **Qualité arabe tunisien** : limite structurelle des modèles gratuits disponibles (Whisper et Piper), documentée par la recherche académique.
- **Espace disque C** : cause identifiée (Docker + pagefile Windows) mais correctif pas encore appliqué (nécessite droits admin + redémarrage).

10.2 Travail restant

- **Dashboard** : interface CRUD documents + statistiques/indicateurs (nb appels selon critère, FAQ, satisfaction) – pas commencé.
- **Docker** : conteneurisation des services + déploiement – dossier vide.
- **Tests arabe approfondis** : lancer `test_language_quality.py` avec de vrais enregistrements et analyser les résultats.
- **Durcissement API REST** : gestion d'erreurs, tests automatisés.

Qualité des données (clôturé) : audit complet de `site_esprit_clean.json` terminé – fiches "gouvernance" et contacts restants corrigés, catalogue RDI hors-sujet supprimé, préfixe d'interface parasite retiré, doublons de contenu fusionnés, conflit des calendriers multi-promotions résolu, grilles tarifaires extraites et intégrées, titres "nom de fichier + page" nettoyés, champ type mort supprimé, détection d'ambiguïté rendue robuste au bruit de score. Base finale : 740 fiches.